API REST · NODE.JS + POSTGRESQL

Balsas Experience API

> RASCUNHO: este documento ainda não está pronto para submissão. Faltam dados acadêmicos e individuais, a URL da API publicada e sua captura de funcionamento.

1. Identificação

Campo	Informação
Instituição	A informar pela equipe
Curso	A informar pela equipe
Disciplina	A informar pela equipe
Professor	A informar pela equipe
Projeto	Balsas Experience API
Data do documento	15/10/2026

2. A equipe

Nome da equipe: **a informar**. Todos os integrantes precisam constar abaixo.

Nome completo	Matrícula	O que fez no projeto
Carinne Borges da Silva	A informar	Contribuição real a confirmar
Júlio César Lima dos Reis	A informar	Contribuição real a confirmar
João Pedro Strasser Santos	A informar	Contribuição real a confirmar
Rogério Mota de Melo	A informar	Contribuição real a confirmar

3. Descrição do projeto

A Balsas Experience API disponibiliza um catálogo de atrações turísticas de Balsas-MA. Permite consultar atrações, filtrar por categoria, tag e texto, acessar detalhes e registrar avaliações de visitantes. Também oferece operações para criar, atualizar e remover atrações.

Serve como fonte de dados para sites e aplicativos voltados a moradores e turistas. O projeto reúne informações dispersas em uma estrutura consultável e persistente: categorias organizam o catálogo, tags permitem cruzar interesses e avaliações registram a experiência dos visitantes.

4. Tecnologias utilizadas

Tecnologia	Versão verificada	Finalidade
Node.js	24.13.1	Ambiente de execução JavaScript
Express	5.2.1	Rotas e respostas HTTP
Prisma CLI / Client	7.10.0	Modelagem, migrações e consultas
PostgreSQL	18.6	Banco de dados relacional
@prisma/adapter-pg	7.10.0	Adaptador Prisma para PostgreSQL
pg	8.23.1	Driver PostgreSQL
dotenv	18.0.5	Carregamento das variáveis de ambiente
npm	11.17.0	Dependências e scripts

5. Modelo de dados

PK = chave primária; FK = chave estrangeira. IDs inteiros são gerados automaticamente; campos sem indicação de opcional são obrigatórios.

Tabela	Campos e restrições
Categoria	id: Int, PK; nome: String, único
Tag	id: Int, PK; nome: String, único
Atracao	id: Int, PK; nome: String; descricao: String; cidade: String (padrão Balsas); estado: String (padrão MA); distanciaCentroKm: Int opcional; criadaEm: DateTime (agora); categoriaId: Int, FK → Categoria.id
Avaliacao	id: Int, PK; visitante: String; nota: Int; comentario: String opcional; criadaEm: DateTime (agora); atracaoId: Int, FK → Atracao.id
_AtracaoToTag	A: Int, FK → Atracao.id; B: Int, FK → Tag.id. PK composta (A, B). Tabela de associação gerenciada pelo Prisma.

Relações: Categoria 1:N Atracao; Atracao 1:N Avaliacao; Atracao N:N Tag. Excluir uma atração remove suas avaliações e associações com tags em cascata; categorias e tags são preservadas. A API valida notas inteiras entre 1 e 5.

Por que separar tags? Uma tabela própria permite reutilizar a mesma tag em várias atrações, evitar duplicação de nomes e aplicar filtros por relacionamento.

6. Endpoints da API

Método	Caminho	O que faz	Status
GET	/	Identificação do serviço	200
GET	/health	Estado do processo HTTP	200
GET	/atracoes	Listagem por nome; categoria e tags incluídas	200
GET	/atracoes?categoria=Lazer &tag=lazer&q=rio	Filtros combinados, sem diferenciar maiúsculas	200
GET	/atracoes/:id	Detalhes, avaliações, média e total	200 / 404
GET	/categorias	Categorias e contagem de atrações	200
POST	/atracoes	Criar atração	201 / 400 / 409
PUT	/atracoes/:id	Substituir dados e tags	200 / 400 / 404 / 409
DELETE	/atracoes/:id	Excluir atração e avaliações	204 / 404
POST	/atracoes/:id/avaliacoes	Criar avaliação de visitante	201 / 400 / 404

Falhas inesperadas podem retornar 500. /health não verifica o banco. POST e PUT exigem nome, descricao e categoria. Cidade, estado, distância e tags são opcionais. PUT substitui tags e restaura os padrões dos campos opcionais omitidos.

6. Endpoints da API · exemplo completo

Requisição e resposta reais do cliente HTTP Node.js/fetch, capturadas em ambiente local em 15/10/2026. O registro temporário foi removido após a coleta.

Requisição

```
POST /atracoes HTTP/1.1
Host: 127.0.0.1:3110
Content-Type: application/json

{
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "categoria": "Histórico e Urbano",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "tags": []
}
```

Resposta

```
HTTP/1.1 201 Created
Location: /atracoes/11
Content-Type: application/json; charset=utf-8

{
  "id": 11,
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "criadaEm": "2026-10-15T16:25:50.047Z",
  "categoriaId": 1,
  "categoria": {
    "id": 1,
    "nome": "Histórico e Urbano"
  },
  "tags": []
}
```

Busca inexistente: `GET /atracoes/2147483647` retorna 404 com `{"erro":"Atração não encontrada"}`. Remoção bem-sucedida retorna 204 sem corpo.

7. Como executar o projeto

Pré-requisitos: Node.js 24 LTS, npm, Git e acesso a um PostgreSQL local ou hospedado. Execute os comandos na raiz do repositório.

1. Clonar e instalar

```
git clone https://github.com/ark-pkr/balsas-experience-api.git
cd balsas-experience-api
npm ci
```

O repositório público está indicado na seção 9.

2. Configurar o ambiente

Copie .env.example para .env e substitua os exemplos pelas credenciais do seu banco. Não publique o .env.

```
DATABASE_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-POOLER/neondb?sslmode=require"
DIRECT_URL="postgresql://USUARIO:SENHA_FICTICIA@HOST-DIRETO/neondb?
sslmode=require&connect_timeout=30"
PORT=3000
```

A aplicação usa DATABASE_URL. O Prisma CLI usa DIRECT_URL ou, se ausente, DATABASE_URL. As senhas acima são fictícias.

3. Gerar cliente, migrar e popular

```
npx prisma generate --config prisma7.config.ts
npx prisma migrate deploy --config prisma7.config.ts
npm run seed
```

Atenção ao seed: apaga os dados existentes antes de inserir oito atrações e três avaliações. Use apenas em banco novo ou descartável; pule essa etapa se o banco já estiver populado.

4. Iniciar e verificar

```
npm start
# Em desenvolvimento, alternativamente: npm run dev
# Em outro terminal:
curl -i http://localhost:3000/health
curl -i http://localhost:3000/atracoes
npm test
```

No PowerShell, use curl.exe. Os testes usam a porta 3108 e criam/removem uma atração temporária. Os 15 cenários passaram na verificação local.

Publicação preparada

O arquivo render.yaml configura um Web Service no Render com Node.js 24.13.1. Build: `npm ci --include=dev && npm run build`; início: `npm start`. Cadastrar DATABASE_URL e DIRECT_URL no painel. O build aplica migrações e não executa seed. A hospedagem fornece PORT.

8. Evidências de funcionamento

Listagem de atrações · HTTP 200

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



BALSAS EXPERIENCE API · CLIENTE HTTP NODE.JS / FETCH

Listagem de atrações

```
GET http://127.0.0.1:3110/atracoes
```

HTTP 200 OK · 694 ms

Resposta recebida · application/json


```
[
  {
    "id": 6,
    "nome": "Cachoeira das Três Marias",
    "descricao": "Formada por três quedas d'água, é uma das atrações naturais mais conhecidas da região.",
    "cidade": "Balsas",
    "estado": "MA",
    "distanciaCentroKm": 50,
    "criadaEm": "2026-10-06T00:39:17.272Z",
    "categoriaId": 3,
    "categoria": {
      "id": 3,
      "nome": "Natureza e Aventura"
    },
    "tags": [
      {
        "id": 12,
        "nome": "aventura"
      },
      {
        "id": 13,
        "nome": "cachoeira"
      },
      {
        "id": 11,
        "nome": "natureza"
      },
      {
        "id": 14,
        "nome": "trilha"
      }
    ]
  }
]
```


Exibindo o primeiro de 8 registros. Resposta integral em registros-http.json.

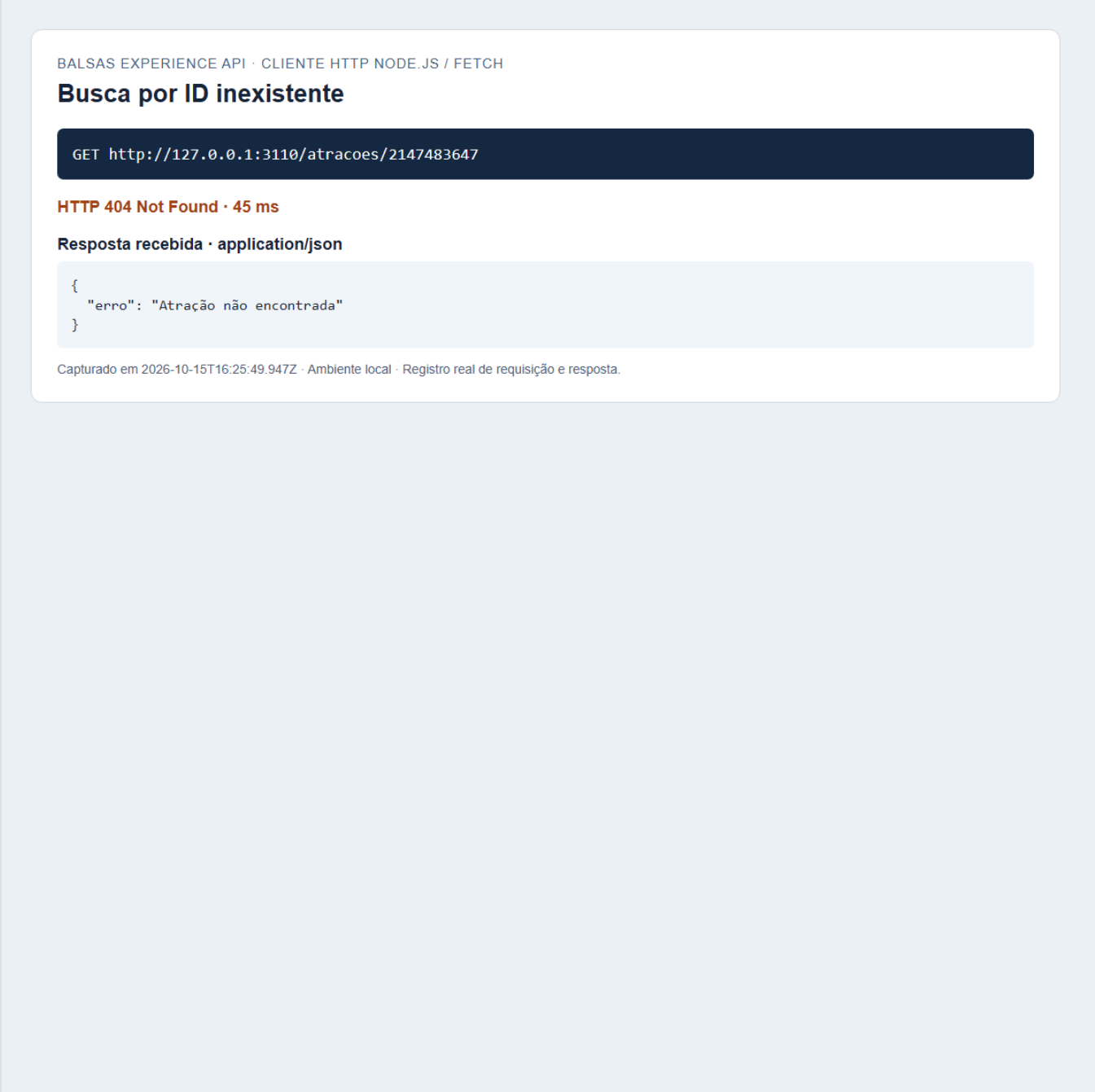
Capturado em 2026-10-15T16:25:49.901Z · Ambiente local · Registro real de requisição e resposta.

GET http://127.0.0.1:3110/atracoes · Status recebido: 200. A captura mostra o primeiro item da listagem; o JSON integral foi preservado.

8. Evidências de funcionamento · continuação

Busca por ID inexistente · HTTP 404

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.

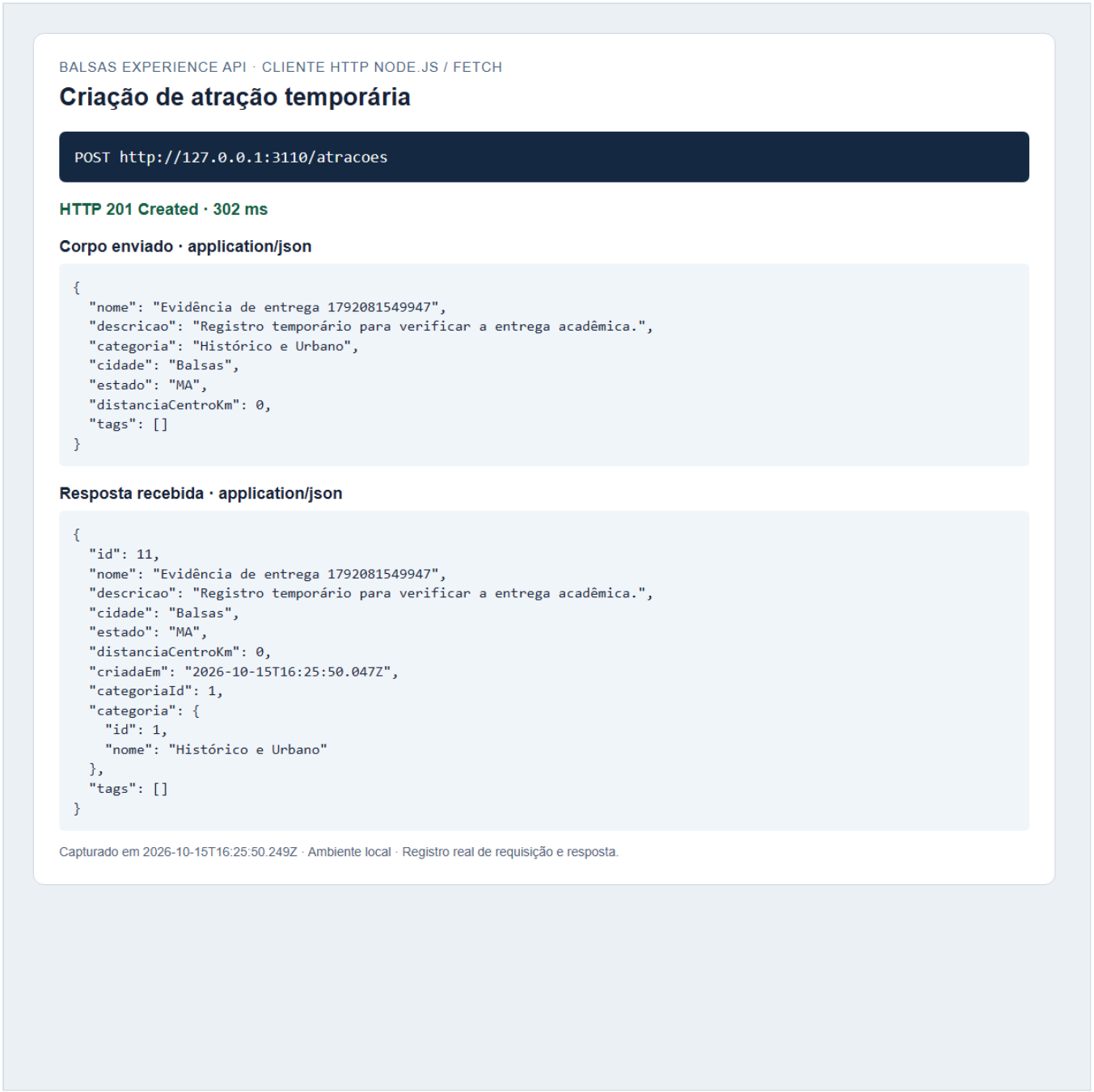


GET http://127.0.0.1:3110/atracoes/2147483647 · Status recebido: 404.

8. Evidências de funcionamento · continuação

Criação de atração temporária · HTTP 201

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



BALSAS EXPERIENCE API · CLIENTE HTTP NODE.JS / FETCH

Criação de atração temporária

```
POST http://127.0.0.1:3110/atracoes
```

HTTP 201 Created · 302 ms

Corpo enviado · application/json


```
{
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "categoria": "Histórico e Urbano",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "tags": []
}
```


Resposta recebida · application/json


```
{
  "id": 11,
  "nome": "Evidência de entrega 1792081549947",
  "descricao": "Registro temporário para verificar a entrega acadêmica.",
  "cidade": "Balsas",
  "estado": "MA",
  "distanciaCentroKm": 0,
  "criadaEm": "2026-10-15T16:25:50.047Z",
  "categoriaId": 1,
  "categoria": {
    "id": 1,
    "nome": "Histórico e Urbano"
  },
  "tags": []
}
```

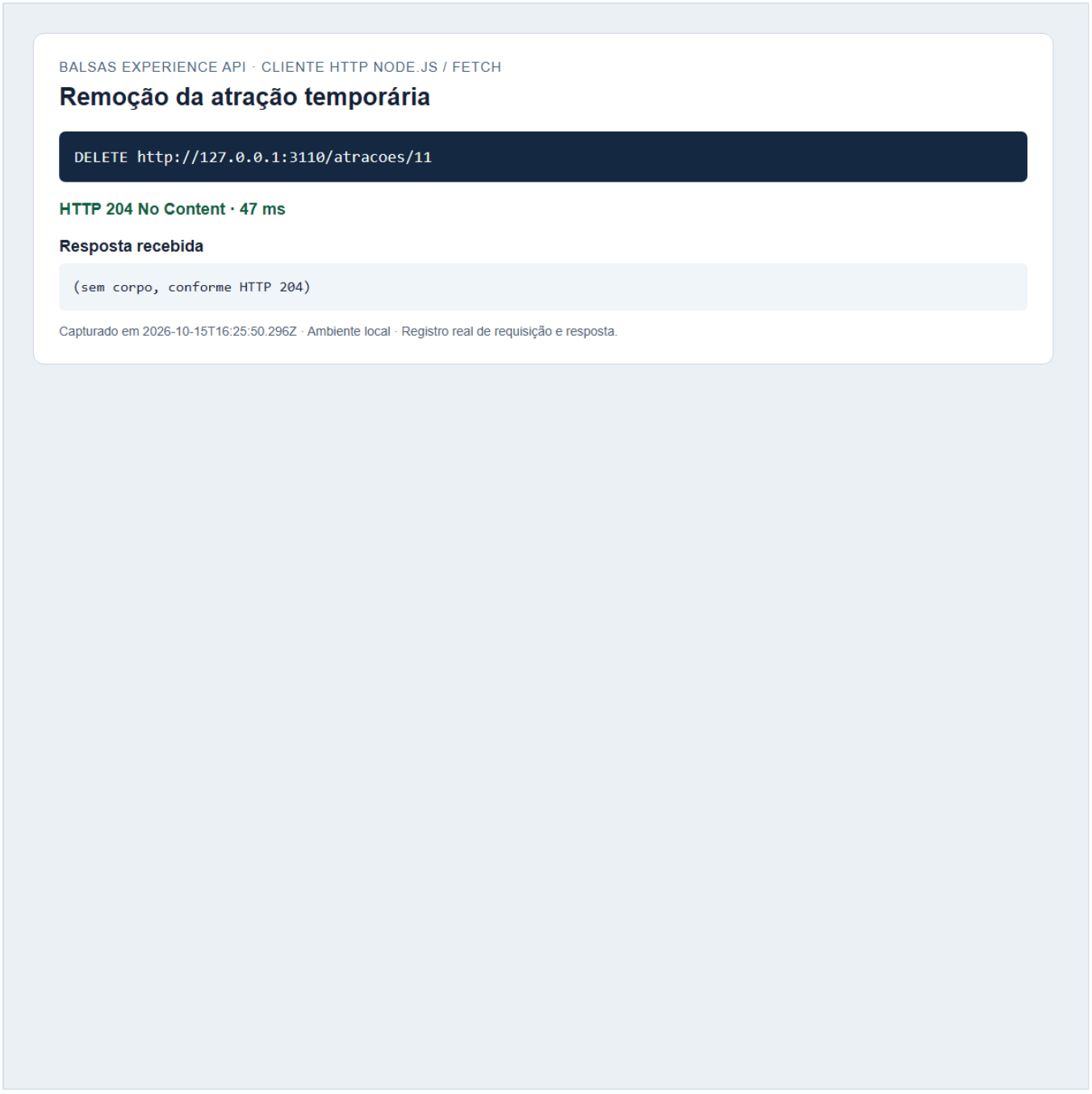

Capturado em 2026-10-15T16:25:50.249Z · Ambiente local · Registro real de requisição e resposta.

POST http://127.0.0.1:3110/atracoes · Status recebido: 201.

8. Evidências de funcionamento · continuação

Remoção da atração temporária · HTTP 204

Captura da visualização do registro real do cliente HTTP Node.js/fetch. Ambiente local, em 15/10/2026. A resposta completa e os cabeçalhos estão em docs/evidencias/registros-http.json.



DELETE http://127.0.0.1:3110/atracoes/11 · Status recebido: 204.

8. Evidências de funcionamento · API publicada

> PENDENTE: informar a URL pública, verificar uma resposta real de /atracoes e inserir a captura. As evidências anteriores comprovam somente a execução local.

9. Links

Destino	Endereço
Repositório público no GitHub	https://github.com/ark-pkr/balsas-experience-api
API publicada	A informar pela equipe

10. Dificuldades e aprendizados

Esta seção depende do relato dos integrantes. Os tópicos abaixo ajudam a registrar a experiência, mas não são apresentados como acontecimentos vividos pela equipe.

Integrante	Dificuldade real	Como resolveu	O que aprendeu
Carinne Borges da Silva	A confirmar	A confirmar	A confirmar
Júlio César Lima dos Reis	A confirmar	A confirmar	A confirmar
João Pedro Strasser Santos	A confirmar	A confirmar	A confirmar
Rogério Mota de Melo	A confirmar	A confirmar	A confirmar

Para cada integrante: descreva um problema concreto, a tentativa que resolveu o problema e o conhecimento adquirido. Exemplos de assuntos que podem ser abordados, se realmente ocorreram: conexão com PostgreSQL, variáveis de ambiente, adaptação ao Prisma 7, relações entre tabelas, migrações, validação de JSON e publicação.

Conceitos técnicos demonstrados pelo projeto

Prisma 7: o cliente usa o adaptador PrismaPg para acessar PostgreSQL. **Modelagem:** tags reutilizáveis são associadas às atrações em uma relação N:N. **Integridade:** a exclusão em cascata evita avaliações sem atração. **HTTP:** criação retorna 201 e Location; remoção retorna 204 sem corpo; registro ausente retorna 404.

Esses conceitos são verificáveis no código; a experiência pessoal de cada integrante será inserida após a confirmação em docs/RELATOS-EQUIPE.md.

Verificação técnica realizada nesta revisão

Os 15 cenários automatizados passaram. Foram coletados registros reais de listagem 200, busca 404, criação 201 e remoção 204, usando o banco PostgreSQL 18.6 configurado. O seed não foi executado nesta revisão.

Antes de enviar pelo portal

1. Preencher instituição, curso, disciplina, professor e todos os integrantes com matrícula e contribuição.
2. Publicar a API e atualizar sua URL no README e neste documento. O repositório público já foi criado.
3. Inserir a captura da API publicada e os aprendizados reais de cada integrante.
4. Gerar novamente o PDF e nomeá-lo balsas-experience-api-equipe-<nome-da-equipe>.pdf.
5. Combinar quem será o único integrante responsável pela submissão e enviar somente o PDF.